

Pipeline Debug (dpipe)

netdev 2.1

Arkadi Sharshevsky <arkadis@mellanox.com>
Jiri Pirko <jiri@mellanox.com>, Matty Kadosh <mattyk@mellanox.com>

Mellanox TECHNOLOGIES

Connect. Accelerate. Outperform.™

Problem Statement

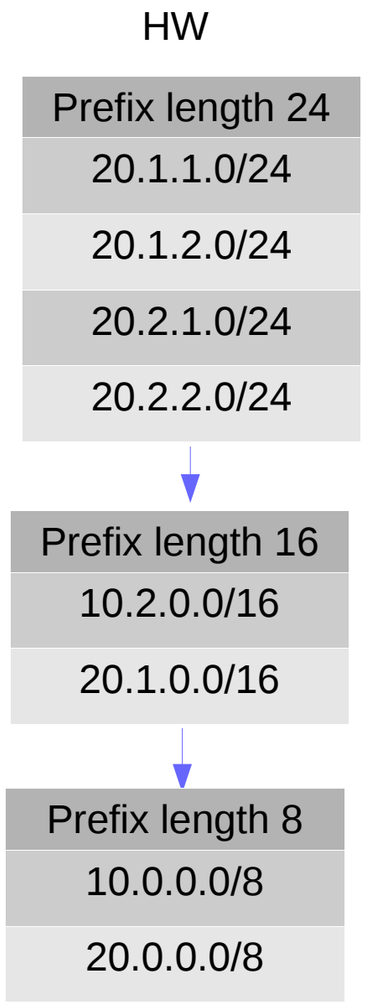
Debugability, visibility

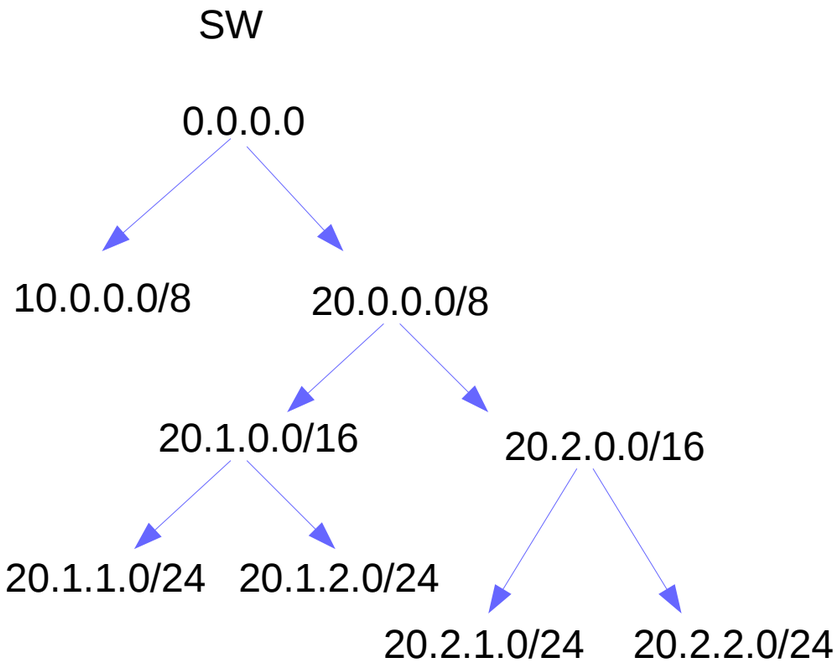
- During offload much of the hardware specifics cannot be presented
- Differences in the software/hardware implementation
  - Performance
  - Scalability
- In case of a failure
  - Limited resources or bad internal resource utilization?
  - Bug due to unmet dependencies?
- Complementary tool needed
  - During system's failure getting the hardware's perspective
  - Debug performance issues

© 2017 Mellanox Technologies

Example LPM

- Implementation can differ
  - SW - trie based lookup
  - HW - link list of hash tables
- In SW Lookup for 10.0.0.0/8 is very fast
- In HW, it can be much slower, adding routes unrelated to 10.0.0.0/8 can make lookup even worse due to adding another link to the list





© 2017 Mellanox Technologies

Problem Statement (continue)

- Hardware switch silicons differences in pipeline, details are different
- Offloading Linux pipeline to different hardware silicons may end with different results
  - Performance
  - Scalability
- In most cases there is more then one way to offload Linux pipeline into single switch silicon
- Vendor neutral hardware abstraction should be provided

© 2017 Mellanox Technologies

Solution

- Model the hardware pipeline as a graph of match/action tables
- Not new approach (P4, Flow-API) for hardware configuration
- New use-case – complementary tool for visibility
  - Response to system configuration via standard tools
- Advantages
  - Hardware specifics can be presented in a generic way
  - Can describe complex hardware blocks
  - Easily extendable

© 2017 Mellanox Technologies

Devlink

- API that allow exposing chip-wide/switch-ASIC-wide stuff
- Introduce new key objects
  - Header - similar to P4, describe packet formats and provide names for fields within the packet and special ASIC metadata
  - Table - describe a generic hardware block which can be modeled by a match/action behavior. Identified by name
- Introduce new commands
  - TABLE_GET - receive general table description
  - HEADERS_GET - receive all supported headers for the device
  - ENTRIES_GET - receive all current entries in specific table
  - COUNTERS_SET - enable/disable counting for specific table

© 2017 Mellanox Technologies

Devlink/Driver Interactions

- Dynamic register/unregister API between driver→Devlink for headers/tables
- Driver local headers – besides protocol headers drivers could define own metadata headers (local enumeration)
- Table operations should be specified by driver at registration (for each table)
  - actions_dump - dump all supported actions
  - matches_dump - dump all supported matches
  - entries_dump - dump all current entries in specific table
  - counters_set_update - alloc/free counters for specific table

© 2017 Mellanox Technologies

Match/Action

- Match/action are used to describe table behavior.
- Match/action are kept close to hardware operations. Match types like LPM are intentionally unsupported
- Example matches
  - field_exact - exact match on a field value of a specific header
  - field_exact_mask - exact match on a field value of a specific header with masking
- Example actions
  - field_modify - set the value of a field in a specific header
  - header_push/pop - add/remove header from packet
  - field_inc/dec - increase/decrease the value of a field in a specific header

© 2017 Mellanox Technologies

Entry

- Specific entry in a table
- Defined by
  - List of match values - can be only subset of the tables matches
  - List of action values - can be only subset of the tables actions
  - Counter – if available
- The entry information can be used to build the table graph dynamically
  - One’s action ↔ another’s match
  - No need to dump extra information about interactions. Everything can be resolved in userspace.

© 2017 Mellanox Technologies

Example L3 Lookup




```
table adjacency {
   size: 4096,
   counters_enabled: true,
   match: { meta.adj_index: exact,
            mata.adj_group_size: exact,
            meta.hash_index: exact }
   action: { ethernet.daddr: set,
            meta.rif_port: set }
}
```

Adjacency

Adj index	Mac addr	Rif port
...		
657	00:01:22:33:44:55	1
658	00:03:22:33:44:55	2
...		

LPM

Prefix length 8

Prefix length 16

IP addr	Group size	Adj index	Rif port
...			
192.162.0.0	2	657	-
192.182.0.0	-	-	2
...			

```
table lpm_prefix_16 {
   size: 4096,
   counters_enabled: true,
   match : { meta.vr_id: exact,
             meta.lpm_prefix: exact,
             ipv4.dst_addr: exact_mask }
   action: {  meta.adj_index: set,
             meta.adj_group_size: set,
             meta.rif_port: set
             meta.lpm_prefix: set }
  }
```

Local Table

Rif port	IP addr	Dst MAC
...		
2	192.182.1.1	01:03:22:33:44:55
...		

```
table local_host {
   size: 4096,
   counters_enabled: true,
   match : { meta.rif_port: exact,
             ipv4.dst_addr: exact}
   action:  { ethernet.daddr: set}
}
```

ERIF

Rif port	Unicast	Broadcast	Multicast	Forward
...				
2		v		v
...				

```
table erif {
   size: 800,
   counters_enabled: true,
   match : { meta.rif_port: exact,
             meta.is_l3_unicast: exact,
             meta.is_l3_broadcat: exact,
             meta.is_l3_multicast: exact }
    action: { meta.l3_forward : set,
             meta.l3_drop : set }
}
```

© 2017 Mellanox Technologies

Real Example ERIF

- First dump supported headers
- Only meta headers are needed for ERIF table
- More headers will be added for next-coming tables (ADJ, Local, LPM)

```
[root@r-mgtswd-1011 devlink]# ./devlink -j -p  dpipe header show pci/0000:03:00.0
{
    "header": {
        "pci/0000:03:00.0": [{
                "name": "mlxsw_meta",
                "field": [{
                        "name": "erif_port",
                        "bitwidth": 32,
                        "mapping_type": "ifindex"
                    },{
                        "name": "l3_forward",
                        "bitwidth": 1
                    },{
                        "name": "l3_drop",
                        "bitwidth": 1
                    }
                ]
            }
        ]
    }
}
```

- Then see the desired table (currently only ERIF)

```
[root@r-mgtswd-1011 devlink]# ./devlink -j -p  dpipe table show pci/0000:03:00.0 name mlxsw_erif
{
    "table": {
        "pci/0000:03:00.0": [{
                "name": "mlxsw_erif",
                "size": 800,
                "counters_enabled": "true",
                "match": [{
                        "type": "field_exact",
                        "header": "mlxsw_meta",
                        "field": "erif_port",
                        "mapping": "ifindex"
                    }
                ],
                "action": [{
                        "type": "field_modify",
                        "header": "mlxsw_meta",
                        "field": "l3_forward"
                    },{
                        "type": "field_modify",
                        "header": "mlxsw_meta",
                        "field": "l3_drop"
                    }
                ]
            }
        ]
    }
}
```

© 2017 Mellanox Technologies

Real Example ERIF (continue)

- Configuration contains two router ports (indexes 0 and 1)
- Corresponding ifindexes

- JSON output can be used by external tool for graph connections and more convenient graphical output

```
[root@r-mgtswd-1011 devlink]# ./devlink -j -p  dpipe table dump pci/0000:03:00.0 name mlxsw_erif
{
    "table_entry": {
        "pci/0000:03:00.0": [{
                "index": 0,
                "counter": 0,
                "match_value": [{
                        "type": "field_exact",
                        "header": "mlxsw_meta",
                        "field": "erif_port",
                        "mapping": "ifindex",
                        "mapping_value": 1025,
                        "value": 0
                    }
                ],
                "action_value": [{
                        "type": "field_modify",
                        "header": "mlxsw_meta",
                        "field": "l3_forward",
                        "value": 1
                    }
                ]
            },{
                "index": 1,
                "counter": 0,
                "match_value": [{
                        "type": "field_exact",
                        "header": "mlxsw_meta",
                        "field": "erif_port",
                        "mapping": "ifindex",
                        "mapping_value": 1023,
                        "value": 1
                    }
                ],
                "action_value": [{
                        "type": "field_modify",
                        "header": "mlxsw_meta",
                        "field": "l3_forward",
                        "value": 1
                    }
                ]
            }
        ]
    }
}
```

© 2017 Mellanox Technologies

Future Extensions

- Too many tables →introduce nesting
  - Hardware is designed in a nested way. Understand better the interactions between big modules L3← → L2.
  - Real ASIC has many components (L2/L3/ACL). Users should be able to “zoom in” on specific block
  - Example of nesting – L3 → VR→LPM→ prefix_16
- Introduce “resource” abstraction
  - Many tables share resources – hash based memory/TCAM/counters
  - Gain debugability of resource related problems
- External tool for table monitoring

© 2017 Mellanox Technologies